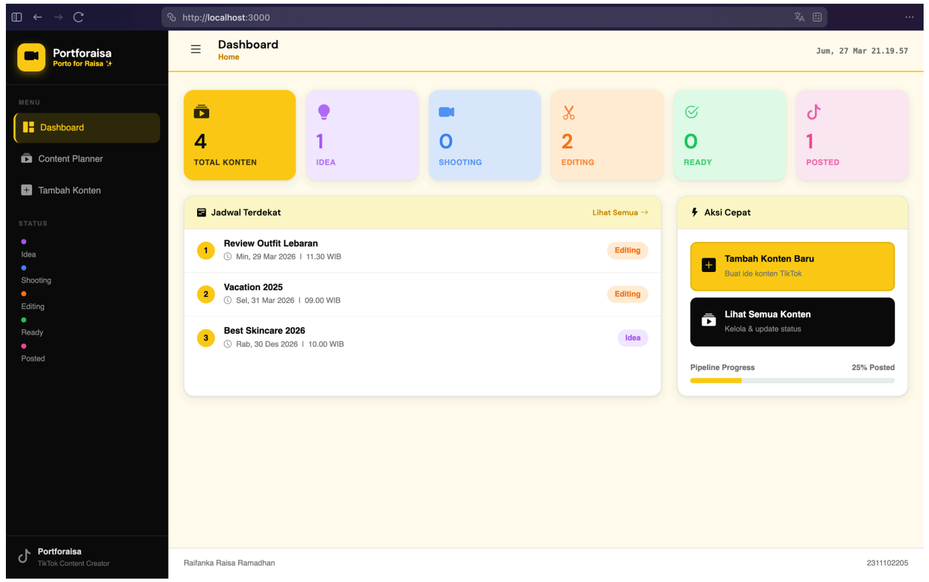
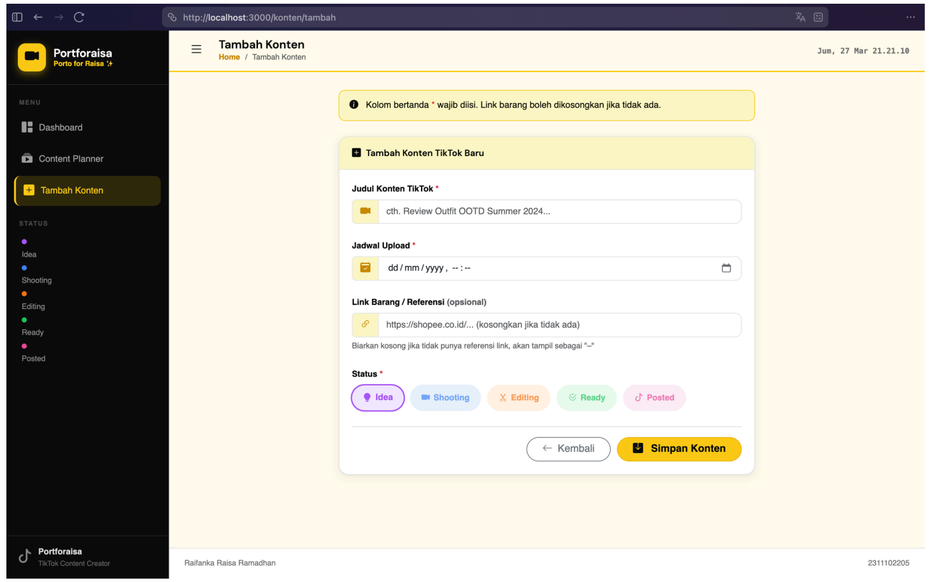
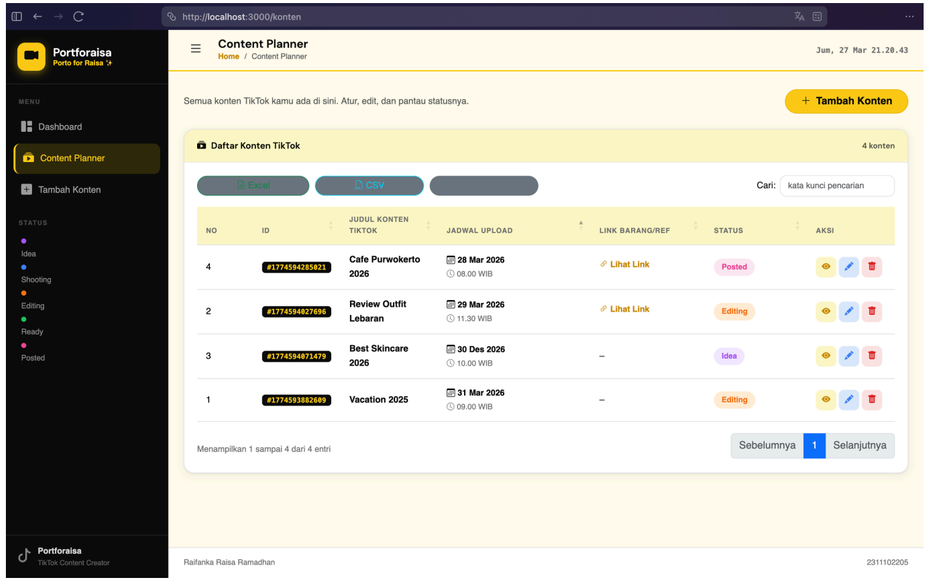
TikTok Content Planner

Website Manajemen Konten Media Sosial TikTok

Raifanka Raisa Ramadhan (2311102205)
S1IF-11-02



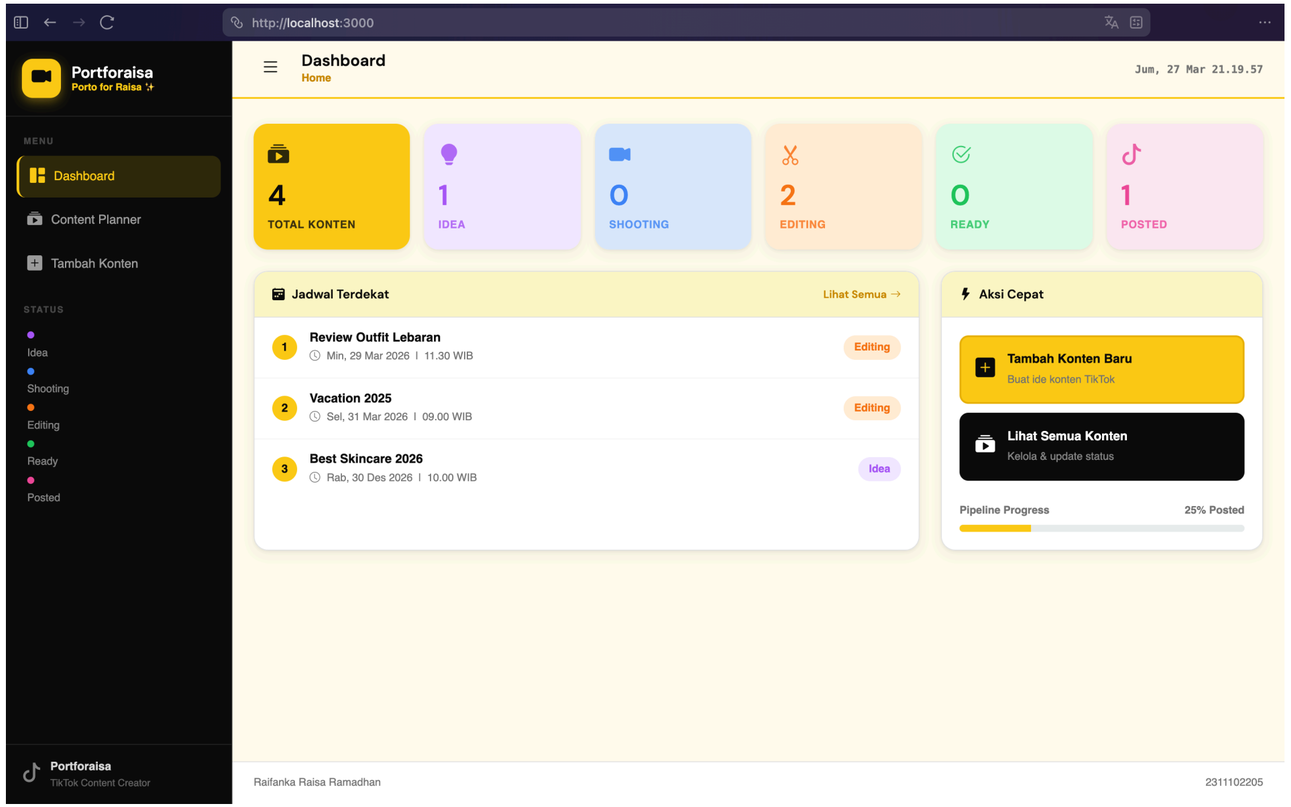




Membangun aplikasi web Portforaisa sebagai solusi Content Planner khusus TikTok dengan fitur CRUD lengkap, menggunakan stack Node.js + Bootstrap 5 + jQuery + DataTables, Fitur Utama yang Dibangun:

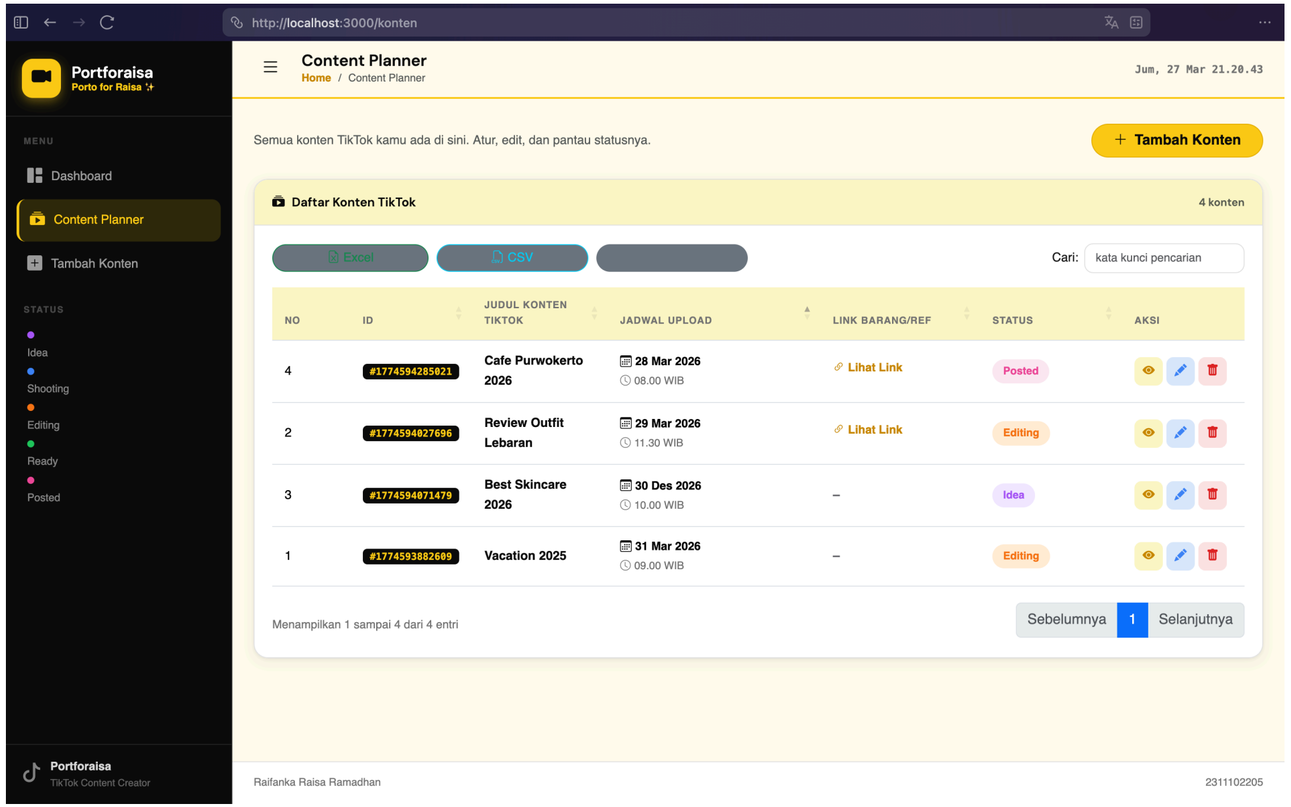
- Dashboard statistik konten
- Tabel data interaktif berbasis JSON (DataTables)
- Form tambah & edit konten dengan validasi jQuery Validate
- Hapus data dengan konfirmasi SweetAlert2
- Status pipeline: Idea → Shooting → Editing → Ready → Posted

Dashboard Page



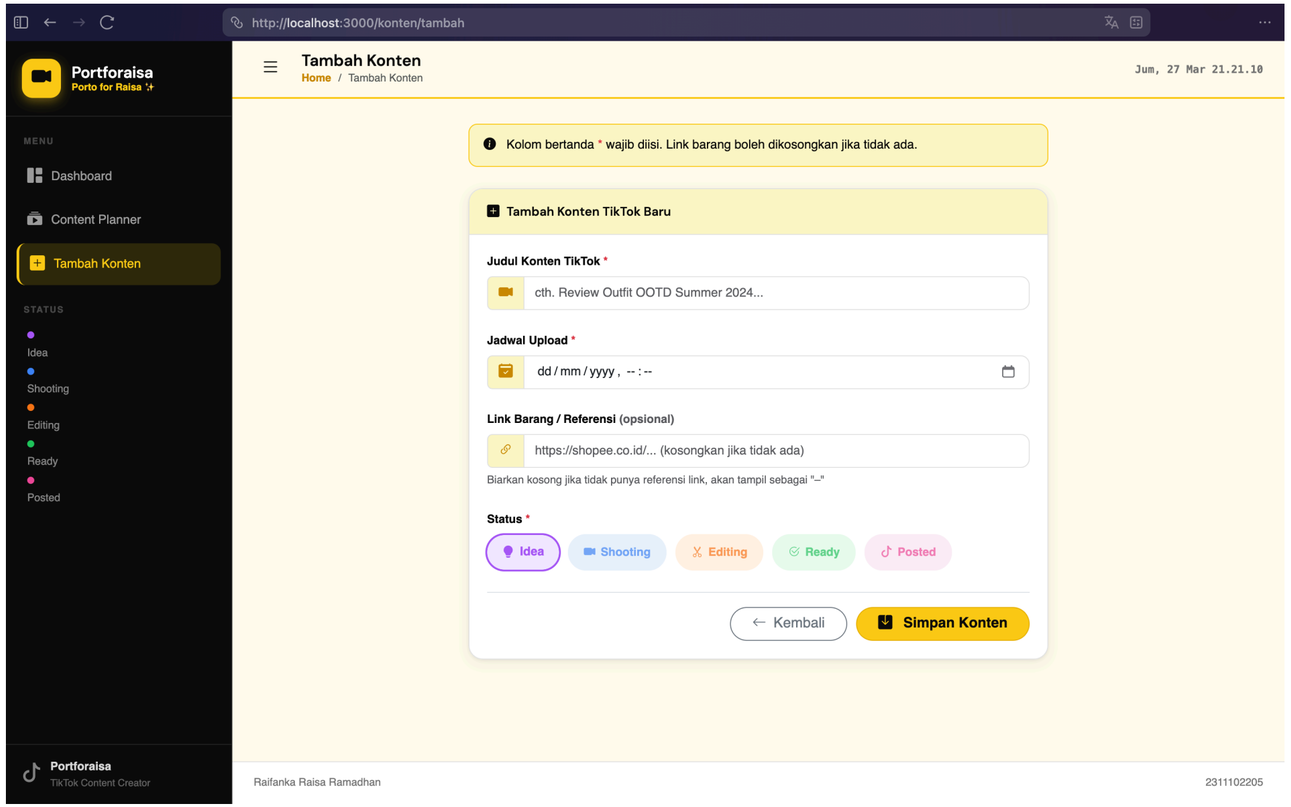
Tampilkan 6 stat cards (Total, Idea, Shooting, Editing, Ready, Posted) dan bagian Jadwal Terdekat. Ini menunjukkan data real dari JSON ditampilkan di UI.

Content Planner Page



Tampilkan tabel DataTables dengan beberapa baris data konten, lengkap dengan tombol aksi, badge status berwarna, dan tombol Export di bagian atas.

Add Konten Page



Tampilkan form input dengan status picker berbentuk pill berwarna.

Struktur Folder dan Teknologi yang Digunakan

```
data
  konten.json
node_modules
public
  css
    style.css
  js
    main.js
routes
  konten.js
views
  konten
    form.ejs
    index.ejs
  partials
    footer.ejs
    header.ejs
  index.ejs
app.js
```

→ Database penyimpanan data

→ Style UI

→ jQuery global utilities

→ Semua endpoint CRUD

→ Halaman Form Tambah/Edit
→ Halaman Tabel Data

→ Script tags & penutup HTML
→ Struktur sidebar & topbar
→ Halaman Dashboard
→ Entry point & konfigurasi server

Pembangunan Website Menggunakan

1. Node.js + Express - Backend server & routing
2. EJS - Template engine (render HTML)
3. Bootstrap - untuk UI Framework & layout responsif
4. jQuery - AJAX, DOM, event handling
5. jQuery Validate - Validasi form sisi client
6. DataTables - Tabel interaktif dari sumber JSON
7. SweetAlert2 - Notif & konfirm hapus

app.js

```js
app.js > ...
const express = require("express");
const bodyParser = require("body-parser");
const path = require("path");
const fs = require("fs");

const kontenRoutes = require("./routes/konten");

const app = express();
const PORT = process.env.PORT || 3000;

app.set("view engine", "ejs");
app.set("views", path.join(__dirname, "views"));

app.use(bodyParser.urlencoded({ extended: true }));
app.use(bodyParser.json());
app.use(express.static(path.join(__dirname, "public")));

const dataPath = path.join(__dirname, "data", "konten.json");

// ── Routes ──────────────────────────────────────────
app.use("/konten", kontenRoutes);

// Dashboard
app.get("/", (req, res) => {
  const raw = fs.existsSync(dataPath)
    ? fs.readFileSync(dataPath, "utf-8")
    : "[]";
  const data = JSON.parse(raw);

  const stats = {
    total: data.length,
    idea: data.filter((k) => k.status === "Idea").length,
    shooting: data.filter((k) => k.status === "Shooting").length,
    editing: data.filter((k) => k.status === "Editing").length,
    ready: data.filter((k) => k.status === "Ready").length,
    posted: data.filter((k) => k.status === "Posted").length,
  };

  // Jadwal terdekat (upcoming, belum posted)
  const now = new Date();
  const upcoming = data
    .filter((k) => k.status !== "Posted" && new Date(k.jadwal) >= now)
    .sort((a, b) => new Date(a.jadwal) - new Date(b.jadwal))
    .slice(0, 5);

  res.render("index", { title: "Dashboard", stats, upcoming });
});

app.listen(PORT, () => {
  console.log(`\n🎬  Portforaisa berjalan di → http://localhost:${PORT}\n`);
});
```

- express() - membuat instance aplikasi web
- bodyParser - membaca data dari form HTML (POST request)
- express.static - menyajikan file CSS, JS, gambar secara statis
- app.use('/konten', kontenRoutes) - semua URL yang diawali /konten ditangani oleh file routes/konten.js
- fs.readFileSync - membaca file konten.json sebagai database
- res.render('index', {...}) - merender halaman EJS dan mengirim data ke view

routes/konten.js

```javascript
const express = require("express");
const router = express.Router();
const fs = require("fs");
const path = require("path");

const dataPath = path.join(__dirname, "..", "data", "konten.json");

function readData() {
  return JSON.parse(fs.readFileSync(dataPath, "utf-8"));
}
function writeData(data) {
  fs.writeFileSync(dataPath, JSON.stringify(data, null, 2));
}
function generateId() {
  return "ktk-" + Date.now();
}

// ── [READ] Halaman tabel ──────────────────────────────
router.get("/", (req, res) => {
  res.render("konten/index", { title: "Content Planner" });
});

// ── [API] JSON untuk DataTables ───────────────────────
router.get("/api/data", (req, res) => {
  const data = readData();
  res.json({ data });
});
```

- <%= ... %> - mencetak nilai variabel sebagai teks di HTML
- <%- ... %> - mencetak nilai mentah (digunakan untuk include partial)
- <% ... %> - menjalankan logika JavaScript (if, forEach, dll) tanpa output
- stats.total, upcoming - variabel yang dikirim dari app.js via res.render()
- k.status.toLowerCase() - mengubah status menjadi huruf kecil untuk dipakai sebagai nama class CSS

jQuery & DataTables

views/konten/index.js

```
columns: [
  // No (urutan tampil)
  {
    data: null,
    render: function(d, t, r, m){ return m.row + 1; },
    orderable: false
  },
  // ID konten
  {
    data: 'id',
    render: function(d){
      var num = d.replace('ktk-','');
      return '<span class="id-badge">#' + num + '</span>';
    }
  },
  // Judul
  {
    data: 'judul',
    render: function(d){
      return '<span class="judul-text">' + d + '</span>';
    }
  },
  // Jadwal Upload
  {
    data: 'jadwal',
    render: function(d){
      if(!d) return '<span class="text-muted">–</span>';
      var dt   = new Date(d);
```

- $(function(){...}) - kode dijalankan setelah seluruh DOM selesai dimuat
- ajax: { url, dataSrc } - DataTables mengambil data langsung dari endpoint JSON, bukan dari HTML
- dataSrc: 'data' - memberitahu DataTables bahwa array data ada di dalam properti bernama data pada JSON response
- render: function(d) - mengatur tampilan tiap kolom secara kustom dengan HTML
- data-id — menyimpan ID konten di atribut HTML, diambil kembali saat tombol diklik via $(this).data('id')
- table.ajax.reload() - memuat ulang data tabel dari server tanpa full page reload
- $('#tblKonten').on('click', '.btn-del', ...) - event delegation, bekerja meskipun baris tabel ditambahkan secara dinamis

jQuery Validate

views/konten/form.js

```
$('#formKonten').validate({
  rules: {
    judul  : { required: true, minlength: 3, maxlength: 200 },
    jadwal : { required: true },
    status : { required: true }
  },
  messages: {
    judul  : { required: 'Judul konten wajib diisi', minlength: 'Minimal 3 karakter' },
    jadwal : { required: 'Jadwal upload wajib diisi' },
    status : { required: 'Pilih status konten' }
  },
  errorPlacement: function(error, element){
    var map = { judul:'errJudul', jadwal:'errJadwal', status:'errStatus' };
    var f   = element.attr('name');
    if(map[f]) $('#' + map[f]).text(error.text()).show();
  },
  highlight   : function(el){ $(el).addClass('is-invalid'); },
  unhighlight : function(el){ $(el).removeClass('is-invalid'); },
  submitHandler: function(form, e){
    e.preventDefault();
    submitForm();
  }
});
```

- rules - mendefinisikan aturan untuk setiap field berdasarkan atribut name
- messages - mengganti teks error default menjadi pesan bahasa Indonesia
- errorPlacement - mengontrol di mana pesan error ditampilkan (bukan di bawah input secara default, tapi di elemen yang sudah ditentukan)
- highlight / unhighlight - menambah atau menghapus class is-invalid dari Bootstrap agar border input berubah merah
- submitHandler - form hanya akan dikirim jika semua aturan validasi terpenuhi; di sini kita ganti pengiriman biasa dengan AJAX

CSS

public/css/style.css

```
:root {
  --yellow: #facc15;
  --yellow-dark: #eab308;
  --yellow-deep: #ca8a04;
  --yellow-soft: #fef9c3;
  --yellow-bg: #fffbeb;
  --dark: #0d0d0d;
  --dark2: #1a1a1a;
  --dark3: #262626;
  --white: #ffffff;
  --grey-100: #f5f5f5;
  --grey-200: #e5e5e5;
  --grey-500: #737373;
  --text: #171717;
  --text-muted: #737373;
  --sidebar-w: 255px;
  --topbar-h: 65px;
  --radius: 16px;
  --radius-sm: 10px;
  --shadow: 0 1px 3px rgba(0, 0, 0, 0.06), 0 4px 12px rgba(0, 0, 0, 0.05);
  --shadow-md: 0 6px 24px rgba(0, 0, 0, 0.1);
  --font: "Poppins", sans-serif;
  --font-head: "DM Sans", sans-serif;

  /* Status colors */
  --idea: #a855f7;
  --shooting: #3b82f6;
  --editing: #f97316;
  --ready: #22c55e;
  --posted: #ec4899;
}
```

- CSS Variables (:root) - warna didefinisikan sekali, lalu dipakai di seluruh file dengan var(--yellow). Jika warna ingin diganti, cukup ubah di satu tempat
- rgba(250, 204, 21, 0.15) - warna kuning dengan transparansi 15% untuk efek highlight menu aktif yang subtle
- backdrop-filter: blur(12px) - efek kaca frosted pada topbar agar terkesan modern
- Status pills menggunakan kelas dinamis dari EJS (sp-<%= k.status.toLowerCase() %>) sehingga warna otomatis menyesuaikan status konten

Terima Kasih